Documentación Técnica – Trabajo Práctico (Segunda Parte)

Sistema: GymControl – Sistema de gestión de gimnasio
Materia: Algoritmo y Estructura de Datos 1 / Programación 1 – UADE
Docentes: Lic. Gustavo E. Escandell – Lic. Facundo Marianelli
Cuatrimestre: 2.º – 2026
Integrantes: Lucas Cragaris (Referente), Tomás van Nynatten, Javier Candalaft, Edney Ribeiro
Repositorio: https://github.com/Lucasalexisc/GymControl

---

1. Introducción y Descripción de las Mejoras (Punto 15.a)

1.1. Problemática

GymControl resuelve la administración diaria de un gimnasio: alta, baja, modificación y consulta de **socios**, **clases** e **inscripciones**, con control de **cupos** por clase, registro de **asistencias**, búsquedas, ordenamientos y reportes estadísticos matriciales. El acceso está restringido a usuarios del personal (recepción, profesores, coordinación, ventas) mediante un inicio de sesión.

Los datos se representan con matrices (listas de listas) definidas en `src/datos.py`:

Matriz	Columnas (constantes de índice)
`usuarios_inicio_sesion`	`INDICE_NOMBRE_USUARIO`, `INDICE_CONTRASENA`, `INDICE_PREGUNTA`, `INDICE_RESPUESTA`
`socios`	`SOCIO_NOMBRE`, `SOCIO_CODIGO`, `SOCIO_EDAD`, `SOCIO_TIPO`, `SOCIO_TELEFONO`
`clases`	`CLASE_CODIGO`, `CLASE_NOMBRE`, `CLASE_NIVEL`, `CLASE_CUPOS`, `CLASE_HORARIO`
`inscripciones`	`INSCRIPCION_CODIGO`, `INSCRIPCION_CODIGO_SOCIO`, `INSCRIPCION_CODIGO_CLASE`, `INSCRIPCION_ASISTENCIAS`, `INSCRIPCION_ESTADO` (1 = activa, 2 = inactiva)

1.2. Nuevas funcionalidades incorporadas

1. **Persistencia de datos en JSON** (`src/persistencia.py`): el estado del gimnasio (socios, clases e inscripciones) se carga al iniciar `main.py` y se guarda automáticamente después de cada alta, baja o modificación. En la Parte 1 los datos vivían solo en memoria y se perdían al cerrar el programa.
2. **Auditoría persistente en archivo de texto** (`src/auditoria.py`): cada acceso (exitoso, fallido o cancelado), cierre de sesión y operación de ABM queda registrado en `auditoria.txt` con número de evento correlativo. La nueva **opción 7 del menú principal** ("Historial de auditoría") muestra el historial y un resumen con la cantidad de eventos por operación.
3. **Horarios de clase** (`src/clases.py`): cada clase incorpora un bloque horario inmutable con días de cursada, hora de inicio y hora de fin, que se pide con `pedir_horario()`, se valida con `son_dias_validos()`, `es_hora_valida()` y `hora_a_minutos()`, y se muestra con `formatear_horario()` en listados, búsquedas y ordenamientos.
4. **Login con método alternativo** (`src/login.py`): `iniciar_sesion()` ofrece ingresar con contraseña (opción 1), con **pregunta de seguridad** (opción 2) o salir (opción 0). Las respuestas se comparan normalizadas con `normalizar_respuesta()` (sin distinguir mayúsculas ni espacios redundantes).
5. **Reportes con conjuntos** (`src/estadisticas.py`), disponibles en el menú de cálculos estadísticos:
   - Opción 5: **socios sin clases activas** (`socios_sin_clases_activas()` / `obtener_socios_sin_clases_activas()`), por diferencia de conjuntos.
   - Opción 6: **disciplinas disponibles sin repetir** (`disciplinas_unicas()` / `obtener_disciplinas_unicas()`).

6. **Suite de pruebas unitarias** (`test_gymcontrol.py`): 12 pruebas sobre 9 funciones del sistema, ejecutables de forma independiente del programa principal.

1.3. Mejoras sobre funcionalidades existentes

- **Liberación de cupos al dar de baja un socio:** `eliminar_afiliados()` ahora llama a `liberar_cupos_de_afiliado()` , que devuelve un cupo a cada clase en la que el socio tenía una inscripción **activa** (las inactivas no devuelven cupo, porque ya lo habían liberado en `baja_inscripcion()`). Corrige el error señalado en la devolución de la Parte 1.
- **Modificación por atributo individual:** `modificar_socio()` y `modificar_clase()` muestran un submenú para elegir qué atributo modificar (nombre, edad, tipo, teléfono / nombre, nivel, cupos, horario), en lugar de pedir todos los campos en secuencia. Solo se guarda y audita si el registro realmente cambió (se compara contra una copia `socio_anterior` / `clase_anterior`).
- **Control de cupos en modificación de inscripciones:** `modificar_inscripcion()` impide reactivar una inscripción si la clase no existe o no tiene cupos, y ajusta los cupos al pasar de activa a inactiva y viceversa.
- **Ordenamientos con `sorted()` :** en `src/ordenamientos.py` y en `buscar_clase_binaria()` se reemplazaron los algoritmos manuales por la función nativa `sorted()` con `key=lambda` , según la indicación del docente.
- **Login sin límite de intentos:** se eliminó el bloqueo tras 3 intentos; el usuario puede reintentar, cambiar de método o salir voluntariamente.
- **Nombres en castellano:** se normalizaron funciones, variables y constantes que estaban en inglés (por ejemplo, `search_position()` → `buscar_posicion()` , `affiliates` → `socios` , `login()` → `iniciar_sesion()`).
- **Modularización:** el archivo único `tp.py` se dividió en módulos dentro de `src/` (ver sección 6).
- **Validación de nombre de socio:** `sumar_afiliados()` y `modificar_socio()` rechazan nombres vacíos o con caracteres que no sean letras, tildes, `ñ` y espacios (reutilizan `es_nombre_clase_valido()`).

---

2. Estructuras de Datos Utilizadas (Punto 15.b)

2.1. Conjuntos (`set`)

Dónde se utiliza:

Archivo	Función	Uso
`src/estadisticas.py`	`obtener_socios_sin_clases_activas(lista_socios=None, lista_inscripciones=None)`	Construye `codigos_todos` (todos los socios) y `codigos_activos` (socios con inscripción en estado 1) y devuelve `codigos_todos - codigos_activos` .
`src/estadisticas.py`	`socios_sin_clases_activas()`	Muestra el resultado anterior ordenado con `sorted()` y la cantidad total.
`src/estadisticas.py`	`obtener_disciplinas_unicas(lista_clases=None)`	Agrega cada `CLASE_NOMBRE` normalizado (`strip().lower()`) a un conjunto `disciplinas` .

Archivo	Función	Uso
`src/estadisticas.py`	`disciplinas_unicas()`	Muestra las disciplinas sin repetir y su cantidad.

Qué información almacena:

- `codigos_todos = {101, 102, ..., 112}` y `codigos_activos = {101, 102, 104, ...}` → resultado: códigos de socios sin clases activas. Por ejemplo, con los datos iniciales el socio `103` (inscripción 303 inactiva) y el `112` (sin inscripciones) forman parte del resultado.
- `disciplinas = {"yoga", "crossfit", "boxeo", "pilates", ...}`: si se cargan dos clases "Yoga" en distintos horarios, la disciplina aparece una sola vez.

Justificación técnica:

- **Unicidad automática:** un socio puede tener varias inscripciones activas; al agregarlo al conjunto `codigos_activos` queda una sola vez sin necesidad de comprobar `if codigo not in lista`.
- **Operación de diferencia:** la pregunta del negocio ("¿qué socios no están en ninguna clase activa?") es exactamente la diferencia `A - B`. Con listas habría que recorrer una lista y buscar cada elemento en la otra con dos bucles anidados.
- **Búsqueda eficiente:** la pertenencia en un `set` se resuelve en tiempo promedio constante, a diferencia de la búsqueda lineal en una lista.
- El orden no importa para el cálculo; cuando se necesita mostrar, se ordena con `sorted()`.

2.2. Tuplas (`tuple`)

Dónde se utiliza:

Archivo	Función / elemento	Uso
`src/datos.py`	Columna `CLASE_HORARIO` de la matriz `clases`	Horario inicial de cada clase.
`src/clases.py`	`pedir_horario()`	Arma `dias = tuple(lista_dias)` y devuelve `(dias, hora_inicio, hora_fin)`.
`src/clases.py`	`formatear_horario(horario)`	Desempaqueta `dias, hora_inicio, hora_fin = horario` para mostrarlo.
`src/clases.py`	`sumar_clase()` / `modificar_clase()` (opción 4)	Guardan la tupla devuelta por `pedir_horario()` en la fila de la clase.
`src/persistencia.py`	`reconstruir_horario(horario)`	Convierte la lista leída desde JSON otra vez en tupla `((dias...), inicio, fin)`.

Qué información representa: el bloque horario de una clase, como tupla anidada:

```
(("Lunes", "Miércoles"), "08:00", "09:00")   # Yoga (201)
(("Sábado",), "10:00", "11:00")             # Stretching (210): tupla de un solo día
```

Por qué se decidió usar una tupla:

- **Inmutabilidad:** días, hora de inicio y hora de fin forman una unidad con sentido propio. No se modifica una parte suelta del horario: cuando cambia, `modificar_clase()` reemplaza **el bloque completo** por una nueva tupla generada con `pedir_horario()`. Esto impide, por ejemplo, cambiar la hora de inicio sin volver a validar que la hora de fin sea posterior.

- **Estructura fija de 3 posiciones:** el orden (días, inicio, fin) es siempre el mismo, lo que permite desempaquetarla directamente en `formatear_horario()` y `reconstruir_horario()`.
- **Diferencia con la fila de la clase:** la fila (`[codigo, nombre, nivel, cupos, horario]`) sí es una lista porque los cupos cambian constantemente; el horario, en cambio, es un dato fijo dentro de ella.

Detalle de persistencia: JSON no tiene tipo tupla, por lo que `json.dump()` guarda el horario como lista. Al recuperar, `cargar_datos()` llama a `reconstruir_horario()` para restaurar la tupla anidada; la prueba unitaria #12 verifica esta reconstrucción.

2.3. Diccionarios (`dict`)

Dónde se utiliza:

Archivo	Diccionario / función	Qué almacena	Finalidad
`src/datos.py`	`NIVELES_CLASE`	`{1: "Principiante", 2: "Intermedio", 3: "Avanzado"}`	Asociar código de nivel con su nombre. Se consulta con `obtener_nombre_nivel()` mediante `.get(codigo, "Desconocido")`.
`src/datos.py`	`TIPOS_SOCIO`	`{1: "Mensual", 2: "Libre", 3: "Premium"}`	Asociar código de tipo de socio con su nombre. Se consulta con `obtener_nombre_tipo()`.
`src/clases.py`	`DIAS_SEMANA`	`{"lunes": "Lunes", "miercoles": "Miércoles", "sabado": "Sábado", ...}`	Organizar información para búsqueda: traduce el día normalizado por `normalizar_dia()` (sin tildes, minúsculas) a su forma correcta. Se usa en `son_dias_validos()` y `pedir_horario()`.
`src/auditoria.py`	`SESION_ACTUAL`	`{"usuario": "admin"}` o `{"usuario": "sin_sesion"}`	Configuración/estado: guarda el usuario autenticado para asociarlo a cada evento de auditoría (`establecer_usuario()`, `registrar_auditoria()`, `cerrar_sesion()`).
`src/auditoria.py`	`resumir_auditoria(eventos)`	`{"ALTA_SOCIO": 2, "BAJA_SOCIO": 1, ...}`	Contabilizar eventos por tipo de operación con `resumen.get(operacion, 0) + 1`. Lo muestra `mostrar_auditoria()`.
`src/persistencia.py`	`guardar_datos()` / `cargar_datos()`	`{"socios": [...], "clases": [...], "inscripciones": [...]}`	Objeto raíz del archivo JSON; permite recuperar cada matriz por su clave.

Justificación técnica:

- **Acceso por clave:** obtener el nombre de un nivel o de un tipo es directo (`NIVELES_CLASE[2]`) sin recorrer una lista ni depender de que el código coincida con una posición.
- **Valor por defecto con `.get()`:** si un código no existe, `obtener_nombre_nivel()` y `obtener_nombre_tipo()` devuelven `"Desconocido"` en lugar de generar un `KeyError`.
- **Conteo:** en `resumir_auditoria()` las claves aparecen a medida que se leen los eventos; con listas habría que mantener dos listas paralelas (operaciones y cantidades) y buscar la posición de cada operación.
- **Estado compartido sin `global`:** `SESION_ACTUAL` es un diccionario mutable del módulo; las funciones modifican su contenido (`SESION_ACTUAL["usuario"] = usuario`) sin necesidad de declarar variables globales.
- **Integración con otras estructuras:** el diccionario raíz del JSON contiene las matrices (listas), que a su vez contienen las tuplas de horario; `DIAS_SEMANA` alimenta la construcción de esas tuplas.

---

3. Persistencia de Datos (Punto 15.c)

Archivo	Formato	Módulo responsable	Información
`datos_gimnasio.json`	JSON (UTF-8)	`src/persistencia.py`	Socios, clases (con horario) e inscripciones.
`auditoria.txt`	Texto plano (UTF-8)	`src/auditoria.py`	Historial de accesos, cierres de sesión y operaciones de ABM.

Ambos archivos se ubican en la **raíz del proyecto** (rutas relativas `RUTA_DATOS_JSON = "datos_gimnasio.json"` y `RUTA_AUDITORIA = "auditoria.txt"`).

3.1. Archivo de texto: `auditoria.txt`

Qué se guarda: un evento por línea, con cuatro campos separados por `" | "`:

```
<identificador> | <usuario> | <operación> | <detalle>
```

Ejemplo real del archivo:

```
EV-0001 | admin | ACCESO_EXITOSO | contraseña
EV-0002 | admin | ALTA_SOCIO | Código=113
EV-0003 | admin | ALTA_INSCRIPCION | Código=312; socio=113; clase=201
EV-0004 | admin | MODIFICACION_INSCRIPCION | Código=312
EV-0005 | admin | BAJA_SOCIO | Código=113; inscripciones eliminadas=1
EV-0006 | admin | CIERRE_SESION | Fin de sesión
```

Operaciones registradas: `ACCESO_EXITOSO` , `ACCESO_FALLIDO` , `ACCESO_CANCELADO` , `CIERRE_SESION` , `ALTA_SOCIO` , `BAJA_SOCIO` , `MODIFICACION_SOCIO` , `ALTA_CLASE` , `BAJA_CLASE` , `MODIFICACION_CLASE` , `ALTA_INSCRIPCION` , `BAJA_INSCRIPCION` , `MODIFICACION_INSCRIPCION` .

Cuándo se escribe: `registrar_auditoria(operacion, detalle, usuario=None, ruta=None)` abre el archivo en modo **agregado** (`"a"`), por lo que el historial crece entre ejecuciones sin borrar lo anterior. Se invoca desde:

- `login.py` → `iniciar_sesion()` (accesos exitosos, fallidos y cancelados).
- `auditoria.py` → `cerrar_sesion()` (llamada desde el bloque `finally` de `ejecutar()` en `main.py`).
- `socios.py` , `clases.py` , `inscripciones.py` → después de cada alta, baja o modificación.

Cuándo se lee: `leer_auditoria(ruta=None)` abre el archivo en modo lectura, separa cada línea con `split(" | ")` y descarta las que no tengan exactamente 4 campos o tengan identificador vacío.

Cómo se reutiliza la información almacenada:

1. **Numeración correlativa:** antes de escribir, `registrar_auditoria()` lee los eventos previos y calcula el siguiente identificador (`EV-0007`, `EV-0008`, ...) a partir de la cantidad de líneas válidas. Así la numeración continúa entre ejecuciones.
2. **Historial y resumen:** `mostrar_auditoria()` (opción 7 del menú principal) muestra todas las líneas y un conteo por operación calculado con `resumir_auditoria()`.

Medidas de integridad: `limpiar_campo(valor)` reemplaza `|` por `/` y elimina saltos de línea y espacios extra de cada campo, para que una entrada del usuario no genere columnas o líneas falsas. Por seguridad, **no se almacenan contraseñas ni respuestas de seguridad**: en los accesos solo se registra el método usado.

3.2. Archivo JSON: `datos_gimnasio.json`

Qué estructuras de Python se serializan: un diccionario con tres claves cuyos valores son las matrices del sistema:

```
{
  "socios": [["Juan Perez", 101, 28, 1, "11-2587-8779"], ...],
  "clases": [[201, "Yoga", 1, 12, [["Lunes", "Miércoles"], "08:00", "09:00"]], ...],
  "inscripciones": [[301, 101, 205, 8, 1], ...]
}
```

Funciones que gestionan la lectura y escritura:

Función	Operación
`guardar_datos(ruta=None)`	Arma el diccionario `datos` y lo escribe con `json.dump(datos, archivo, ensure_ascii=False, indent=2)`. `ensure_ascii=False` conserva tildes y `ñ`; `indent=2` deja el archivo legible. Devuelve `True` o `False`.
`cargar_datos(ruta=None)`	Lee con `json.load()`, valida la estructura (objeto raíz, tres listas, filas de clase de 5 columnas), reconstruye las tuplas de horario y reemplaza el contenido de las matrices. Devuelve `True` o `False`.
`reconstruir_horario(horario)`	Convierte la lista `[[días], inicio, fin]` leída del JSON en la tupla `((días), inicio, fin)`; lanza `ValueError` si el formato es inválido.

Cómo se garantiza la persistencia entre ejecuciones:

1. **Generar o modificar información:** el usuario realiza un alta, baja o modificación (por ejemplo, `sumar_afiliados()` agrega un socio).
2. **Guardar:** la misma función llama inmediatamente a `guardar_datos()`, que sobrescribe `datos_gimnasio.json` con el estado completo. No depende de que el usuario salga correctamente del menú.
3. **Finalizar la ejecución:** el programa termina (opción 0, o incluso por interrupción).
4. **Volver a ejecutar:** lo primero que hace `ejecutar()` en `main.py` es llamar a `cargar_datos()`, antes del login.
5. **Recuperar:** `cargar_datos()` lee el JSON y actualiza las matrices **en el lugar** con asignación por rebanada (`socios[:] = nuevos_socios`, `clases[:] = clases_reconstruidas`, `inscripciones[:] = nuevas_inscripciones`). Esto es importante porque todos los módulos importan las mismas listas desde `datos.py`; reemplazar su contenido (en lugar de reasignar el nombre) hace que todos vean los datos recuperados.

Primera ejecución: si `datos_gimnasio.json` no existe, `cargar_datos()` captura `FileNotFoundError` y llama a `guardar_datos()` para crearlo a partir de los datos iniciales de `datos.py`.

Conversión Python ↔ JSON: listas, enteros y cadenas se convierten directamente. La única pérdida de tipo es la tupla de horario (JSON solo tiene arreglos), y se resuelve con `reconstruir_horario()` al cargar.

---

4. Manejo de Excepciones y Validaciones (Punto 15.d)

4.1. Situaciones controladas con `try / except`

Se capturan siempre **excepciones específicas** (no se usa `except:` genérico como en la Parte 1), y cada bloque informa al usuario o devuelve un valor que permite continuar.

Situación	Función (archivo)	Excepción capturada	Respuesta del sistema
Ingreso de texto donde se espera un número (conversión de tipos)	`es_entero()` (`validaciones.py`)	`ValueError`, `TypeError`	Devuelve `False`; `pedir_entero()` muestra el mensaje de error recibido y vuelve a pedir el dato.
Conversión a decimal inválida	`es_flotante()` (`validaciones.py`)	`ValueError`, `TypeError`	Devuelve `False`.
Archivo JSON inexistente (primera ejecución)	`cargar_datos()` (`persistencia.py`)	`FileNotFoundError`	Crea el archivo con los datos iniciales mediante `guardar_datos()`.
JSON corrupto, con claves faltantes, tipos incorrectos o filas mal formadas	`cargar_datos()` (`persistencia.py`)	`json.JSONDecodeError`, `KeyError`, `ValueError`, `TypeError`	Muestra "*Aviso: no se pudo cargar datos_gimnasio.json; se utilizarán los datos en memoria.*" y continúa con los datos iniciales.
Horario con formato inválido dentro del JSON	`reconstruir_horario()` (`persistencia.py`)	Lanza `ValueError` (con `raise`)	La excepción la captura `cargar_datos()` (fila anterior).
Archivo JSON sin permisos o con problemas de codificación	`cargar_datos()` (`persistencia.py`)	`OSError`, `UnicodeError`	Mismo aviso; el programa sigue funcionando.
Error al escribir el JSON (disco, permisos, dato no serializable)	`guardar_datos()` (`persistencia.py`)	`OSError`, `TypeError`	Muestra "*Aviso: no se pudieron guardar los datos en datos_gimnasio.json.*" y devuelve `False`.
Archivo de auditoría inexistente	`leer_auditoria()` (`auditoria.py`)	`FileNotFoundError`	Devuelve lista vacía `[]` (no hay eventos todavía; el primer evento será `EV-0001`).
Archivo de auditoría ilegible	`leer_auditoria()` (`auditoria.py`)	`OSError`, `UnicodeError`	Muestra "*Aviso: no se pudo leer auditoria.txt.*" y devuelve `None`; `mostrar_auditoria()` no muestra nada.
Error al escribir un evento de auditoría	`registrar_auditoria()` (`auditoria.py`)	`OSError`	Muestra "*Aviso: no se pudo guardar el evento en auditoria.txt.*" y devuelve `False`; la operación del usuario no se interrumpe.

Situación	Función (archivo)	Excepción capturada	Respuesta del sistema
Interrupción de la entrada (Ctrl+C o fin de entrada)	`ejecutar()` (`main.py`)	`EOFError`, `KeyboardInterrupt`	Muestra "*Entrada interrumpida. Saliendo del sistema.*"; el bloque `finally` ejecuta `cerrar_sesion()` para registrar el cierre en la auditoría.

4.2. Validaciones sin excepciones (control previo)

Estas situaciones se previenen con condiciones antes de que se produzca un error:

Situación	Dónde se controla	Respuesta
Opción inexistente en menús	`pedir_opcion_principal()`, `pedir_opcion_socios()`, `pedir_opcion_clases()`, `pedir_opcion_inscripciones()`, `menu_busqueda()`, `menu_ordenamiento()`, `pedir_opcion_matriz()` (`menus.py`)	"*La opción ingresada es errónea...*" y se vuelve a mostrar el menú.
Opción inválida en el login	`iniciar_sesion()` (`login.py`)	"*Opción inválida. Elija 1, 2 o 0.*"
Credenciales incorrectas / usuario inexistente	`iniciar_sesion()` (`login.py`)	"*Credenciales incorrectas...*", se registra `ACCESO_FALLIDO` y se permite reintentar o cambiar de método.
Respuesta de seguridad vacía	`iniciar_sesion()` (`login.py`)	Se considera inválida (`bool(credencial)`).
Valores fuera de rango (edad 0–100, tipo 1–3, nivel 1–3, cupos ≥ 1, estado 1–2, asistencias ≥ 0)	`pedir_entero()` con `minimo` / `maximo` (`validaciones.py`)	Mensaje específico y nuevo ingreso.
Nombre vacío o con caracteres no permitidos	`es_nombre_clase_valido()` (regex) en `sumar_afiliados()`, `modificar_socio()`, `sumar_clase()`, `modificar_clase()`	Mensaje de error y nuevo ingreso.
Teléfono con formato distinto de `XX-XXXX-XXXX`	`es_telefono_valido()` (regex `\d{2}-\d{4}-\d{4}`)	Mensaje de error y nuevo ingreso.
Día de cursada inexistente	`son_dias_validos()` + `DIAS_SEMANA` (`clases.py`)	Mensaje de error y nuevo ingreso.
Hora con formato inválido o fuera de rango (`24:00`, `12:60`)	`es_hora_valida()` (`clases.py`)	Mensaje de error y nuevo ingreso.
Hora de fin anterior o igual a la de inicio	`pedir_horario()` con `hora_a_minutos()` (`clases.py`)	"*ERROR, ingrese una hora posterior a la de inicio...*"

Situación	Dónde se controla	Respuesta
Socio, clase o inscripción inexistente	`buscar_posicion_socio()`, `buscar_posicion_clase()`, `buscar_posicion_inscripcion()`, `existe_codigo_socio()`, `existe_codigo_clase()` (`utilidades.py`)	*"Socio no encontrado."*, *"Clase no encontrada."*, *"Inscripción no encontrada."* o nuevo ingreso del código.
Clase sin cupos disponibles	`alta_inscripcion()` (`inscripciones.py`)	*"No hay cupos disponibles para esa clase."*
Inscripción activa duplicada	`socio_esta_inscripto_en_clase()` en `alta_inscripcion()`	*"ERROR: El afiliado ya se encuentra inscripto en esta clase."*
Baja de una inscripción ya finalizada (estado del sistema)	`baja_inscripcion()`	*"La inscripción ya estaba finalizada."*
Reactivar una inscripción sin cupo o de una clase inexistente	`modificar_inscripcion()`	*"No hay cupos disponibles para activar esta inscripción."* / *"No se puede activar porque la clase no existe."*
Referencias a socios o clases eliminados en listados	`listar_inscripciones()`, `baja_inscripcion()`, `modificar_inscripcion()`, `ordenar_inscripciones_por_asistencias()`	Se muestra *"Socio inexistente"* / *"Clase inexistente"* en lugar de fallar con un índice inválido.
Confirmación de bajas	`eliminar_afiliados()`, `eliminar_clase()`, `baja_inscripcion()`	Si no se responde `s`, *"Operación cancelada."*

5. Pruebas Unitarias (Punto 15.e)

5.1. Ejecución independiente

Las pruebas están en `test_gymcontrol.py`, en la raíz del proyecto. No usan clases (`class`) ni librerías externas: cada prueba es una función que llama a una función real del sistema y compara el resultado obtenido con el esperado mediante `mostrar_resultado_prueba()`, que imprime los 5 puntos exigidos por la consigna (función probada, datos utilizados, resultado esperado, resultado obtenido y estado APROBADA/FALLIDA).

Desde la raíz del proyecto:

```
python test_gymcontrol.py        # Windows
python3 -B test_gymcontrol.py    # Linux / macOS (-B evita generar __pycache__)
```

`ejecutar_pruebas()` recorre la lista de funciones de prueba, cuenta las aprobadas e imprime al final `RESUMEN FINAL: 12/12 pruebas aprobadas.` El archivo no ejecuta el menú ni el login, por lo que funciona sin intervención del usuario.

5.2. Tabla resumen

N°	Categoría	Función testeada	Datos de entrada	Resultado esperado	Qué situación verifica

N°	Categoría	Función testeada	Datos de entrada	Resultado esperado	Qué situación verifica
1	Caso válido	`es_telefono_valido()`	`"11-2587-8779"`	`True`	Un teléfono con el formato `XX-XXXX-XXXX` es aceptado.
2	Caso válido	`es_nombre_clase_valido()`	`"Funcional Avanzado"`	Coincidencia (no `None`) → `True`	Un nombre de varias palabras con espacios es válido.
3	Caso válido	`socio_esta_inscripto_en_clase()`	socio `101`, clase `205` (inscripción 301 activa)	`True`	Se detecta una inscripción activa existente (base del bloqueo de duplicados).
4	Caso válido	`resumir_auditoria()`	3 eventos: 2 `ALTA_SOCIO`, 1 `BAJA_SOCIO`	`{"ALTA_SOCIO": 2, "BAJA_SOCIO": 1}`	El diccionario cuenta correctamente los eventos por operación.
5	Caso inválido	`es_telefono_valido()`	`"1125878779"` (sin guiones)	`False`	Se rechaza un teléfono con formato incorrecto.
6	Caso inválido	`es_entero()`	`"abc"`	`False`	La conversión fallida (`ValueError`) se controla y no detiene el programa.
7	Caso inválido	`socio_esta_inscripto_en_clase()`	socio `103`, clase `201` (inscripción 303 con estado 2)	`False`	Una inscripción inactiva no cuenta como inscripción vigente, por lo que el socio puede volver a inscribirse.

N°	Categoría	Función testeada	Datos de entrada	Resultado esperado	Qué situación verifica
8	Caso límite	`es_hora_valida()`	`"00:00"`, `"23:59"`, `"24:00"`	`(True, True, False)`	Bordes del rango horario: mínimo y máximo válidos y primer valor fuera de rango.
9	Caso límite	`liberar_cupos_de_afiliado()`	socio `999` sin inscripciones; clase 201 con cupo 12	`(0, 12)`	Dar de baja un socio sin inscripciones no libera cupos ni altera la clase.
10	Específico del dominio	`liberar_cupos_de_afiliado()`	socio `112` con inscripción activa en 201 (cupo 14) e inactiva en 203 (cupo 15)	`(1, 15, 15)`	Corrección del error de la Parte 1: solo la inscripción activa devuelve cupo; la inactiva no duplica la liberación.
11	Específico del dominio (conjuntos)	`obtener_socios_sin_clases_activas()`	socios `{101, 103, 112}`; 101 activa, 103 inactiva	`{103, 112}`	La diferencia de conjuntos identifica socios sin clases activas, incluidos los que solo tienen inscripciones inactivas.
12	Específico del dominio (JSON + tuplas)	`guardar_datos()` / `cargar_datos()`	Se guarda el estado, se altera en memoria el cupo de la clase 201 a 99 y se recarga	`(True, True, 12, True)`	La información se recupera desde el JSON (el cupo vuelve a 12) y el horario se reconstruye como tupla anidada.

Distribución respecto de la consigna: 4 casos válidos, 3 inválidos, 2 límite y 3 específicos del dominio (la consigna exige al menos 1). En total 12 pruebas sobre **9 funciones distintas**: `es_telefono_valido`, `es_nombre_clase_valido`, `socio_esta_inscripto_en_clase`, `resumir_auditoria`, `es_entero`, `es_hora_valida`, `liberar_cupos_de_afiliado`, `obtener_socios_sin_clases_activas` y el par `guardar_datos` / `cargar_datos`.

5.3. Decisiones de diseño de las pruebas

- **Independencia de los datos reales:** las pruebas 4, 9, 10 y 11 usan matrices propias (`inscripciones_prueba` , `clases_prueba` , `socios_prueba`). Por eso `liberar_cupos_de_afiliado()` recibe las matrices como parámetros y `obtener_socios_sin_clases_activas()` acepta listas opcionales: se pueden probar sin modificar los datos del gimnasio.
- **Restauración del estado en la prueba 12:** antes de probar, se hace una copia (`respaldo_socios` , `respaldo_clases` , `respaldo_inscripciones`); el bloque `finally` restaura las matrices y vuelve a guardar el JSON, de modo que ejecutar las pruebas no altera los datos del sistema.
- **Qué pasa si se modifican los datos (para la defensa):**
  - Prueba 3: si se usa la clase `201` con el socio `101` , el resultado pasa a `False` porque no existe esa inscripción.
  - Prueba 8: `"7:00"` también devuelve `True` (la función acepta horas de un dígito); `"23:60"` devuelve `False` .
  - Prueba 10: si ambas inscripciones estuvieran activas, se esperaría `(2, 15, 16)` .
  - Prueba 11: si se agrega una inscripción activa para el socio `112` , el resultado esperado pasa a ser `{103}` .

---

6. Cambios Respecto de la Primera Parte (Punto 15.f)

6.1. Qué se mantuvo de la Parte 1

- La **problemática** (gestión de gimnasio) y las tres entidades: socios, clases e inscripciones.
- La representación con **matrices** y constantes de índice para acceder a cada columna.
- Los datos iniciales de ejemplo (12 socios, 10 clases, 11 inscripciones, 10 usuarios).
- Los ABM de socios, clases e inscripciones, con confirmación de bajas.
- **Búsqueda binaria** de clases (`buscar_clase_binaria()`) y **secuencial** de socios (`buscar_socio_secuencial()`).
- Los **reportes matriciales**: `socios_por_clase()` , `matriz_inscripciones_por_nivel_clase()` , `total_asistencias_por_clase()` y `matriz_socios_por_tipo_y_clase()` .
- Uso de `map` , `filter` , `reduce` (suma de asistencias en `total_asistencias_por_clase()`), funciones `lambda` y **expresiones regulares** (nombres y teléfono).
- La función genérica de búsqueda con `filter` y `lambda` (hoy `buscar_posicion()`).
- El bloqueo de inscripciones activas duplicadas y la validación de teléfono.

6.2. Errores corregidos y reestructuración

Aspecto de la Parte 1	Problema	Solución en la Parte 2
Baja de socio	No se liberaba el cupo de las clases en las que el socio estaba inscripto (señalado en la devolución).	`eliminar_afiliados()` llama a `liberar_cupos_de_afiliado()` antes de eliminar las inscripciones. Verificado por las pruebas 9 y 10.
`try / except`	Se usaba `except:` genérico, un tema que todavía no se había visto.	Se capturan excepciones específicas (`ValueError` , `TypeError` , `FileNotFoundError` , `OSError` , etc.) solo donde pueden ocurrir (ver sección 4).
Modificación de socios y clases	Se pedían todos los atributos en secuencia (escribiendo `si` para mantener cada valor).	Submenú para elegir un atributo por vez en `modificar_socio()` y `modificar_clase()` .
Ordenamientos	Algoritmos implementados a mano (por ejemplo, selección en `ordenar_socios_por_edad()`).	Uso de `sorted()` con `key=lambda` en `ordenamientos.py` y `buscar_clase_binaria()` .

Aspecto de la Parte 1	Problema	Solución en la Parte 2
Login	Bloqueo después de 3 intentos.	Sin límite de intentos, con método alternativo por pregunta de seguridad y salida voluntaria.
Idioma del código	Mezcla de nombres en inglés y castellano (`affiliates`, `classes`, `enrollments`, `search_position()`, `get_level_name()`, `login()`).	Todo normalizado al castellano (`socios`, `clases`, `inscripciones`, `buscar_posicion()`, `obtener_nombre_nivel()`, `iniciar_sesion()`).
Organización	Un único archivo `tp.py` con todas las funciones.	Modularización en `src/` con responsabilidades separadas.
Datos volátiles	Todo se perdía al cerrar el programa.	Persistencia en `datos_gimnasio.json` y `auditoria.txt`.

Estructura modular actual:

Módulo	Responsabilidad
`main.py`	Punto de entrada: carga los datos, inicia sesión, abre el menú principal y garantiza el cierre de sesión.
`src/datos.py`	Matrices, constantes de índice y diccionarios de niveles y tipos.
`src/login.py`	Autenticación por contraseña o pregunta de seguridad.
`src/menus.py`	Menús y submenús; solo coordinan las llamadas.
`src/socios.py`	ABM de socios, eliminación de inscripciones asociadas y liberación de cupos.
`src/clases.py`	ABM de clases y gestión de horarios. (El nombre se refiere a las clases del gimnasio; **no** se usan clases de Python.)
`src/inscripciones.py`	Alta, baja, modificación y listados de inscripciones.
`src/busquedas.py`	Búsqueda binaria y secuencial.
`src/ordenamientos.py`	Listados ordenados con `sorted()`.
`src/estadisticas.py`	Reportes matriciales y reportes con conjuntos.
`src/utilidades.py`	Búsquedas de posición y verificaciones de existencia reutilizables.
`src/validaciones.py`	Validación de enteros, nombres y teléfonos; lectura validada con `pedir_entero()`.
`src/persistencia.py`	Guardado y carga del JSON.
`src/auditoria.py`	Escritura, lectura y resumen del archivo de auditoría.
`test_gymcontrol.py`	Pruebas unitarias.

6.3. Qué es nuevo en la Segunda Parte

- **Módulos nuevos:** `src/persistencia.py`, `src/auditoria.py` y `test_gymcontrol.py` (el resto surge de dividir `tp.py`).
- **Archivos de datos nuevos:** `datos_gimnasio.json` y `auditoria.txt`.
- **Funcionalidades nuevas:** persistencia automática; historial de auditoría (opción 7); horarios de clase como tuplas; login con pregunta de seguridad; reportes de socios sin clases activas y disciplinas únicas (opciones 5 y 6 del menú estadístico); liberación de cupos al eliminar socios.

- **Nuevas estructuras:** conjuntos en `estadisticas.py`; tuplas de horario en `clases.py` y `persistencia.py`; diccionarios `DIAS_SEMANA`, `SESION_ACTUAL`, el conteo de `resumir_auditoria()` y el objeto raíz del JSON (los diccionarios `NIVELES_CLASE` y `TIPOS_SOCIO` reemplazan la traducción de códigos de la Parte 1).
- **Nuevas columnas:** `CLASE_HORARIO` en la matriz `clases`, y `INDICE_PREGUNTA` / `INDICE_RESPUESTA` en `usuarios_inicio_sesion`.

---

7. Guía de Ejecución (Punto 16)

7.1. Requisitos previos

- **Python 3.9 o superior** (probado con Python 3.10).
- **No requiere instalar librerías.** Solo se usan módulos incluidos en Python y trabajados en la materia:

Módulo	Dónde	Para qué
`json`	`src/persistencia.py`	Leer y escribir `datos_gimnasio.json`.
`re`	`src/validaciones.py`	Expresiones regulares para nombres y teléfonos.
`functools.reduce`	`src/estadisticas.py`	Suma de asistencias por clase.
`sys`	`main.py`, `test_gymcontrol.py`	Agregar la carpeta `src` a la ruta de importación (`sys.path.insert(0, "src")`).

7.2. Pasos para ejecutar el sistema

1. Descargar o clonar el repositorio:

```
git clone https://github.com/Lucasalexisc/GymControl.git
```

2. Abrir una terminal **en la carpeta raíz del proyecto** (donde están `main.py` y `datos_gimnasio.json`):

```
cd GymControl
```

3. Ejecutar:

```
python main.py      # Windows
python3 main.py     # Linux / macOS
```

4. Iniciar sesión con las credenciales de demostración:
   - Opción **1 (contraseña):** usuario `admin`, contraseña `admin1234`.
   - Opción **2 (pregunta de seguridad):** usuario `admin`, pregunta "*¿Cuál es tu color favorito?*", respuesta `azul`.
   - Los demás usuarios están en `usuarios_inicio_sesion` (`src/datos.py`).

> **Importante:** el programa debe ejecutarse desde la raíz del proyecto. Las rutas `datos_gimnasio.json`, `auditoria.txt` y `src` son relativas a la carpeta desde la que se ejecuta el comando; si se ejecuta desde otra carpeta, Python no encontrará los módulos de `src` y los archivos de datos se crearían en otro lugar.

Si `datos_gimnasio.json` no existe, se crea automáticamente con los datos iniciales; si `auditoria.txt` no existe, se crea con el primer evento registrado.

7.3. Pasos para ejecutar las pruebas

Desde la misma carpeta raíz:

```
python test_gymcontrol.py        # Windows
python3 -B test_gymcontrol.py    # Linux / macOS
```

Salida esperada al final: `RESUMEN FINAL: 12/12 pruebas aprobadas.`

7.4. Verificación rápida de la persistencia (para la defensa)

1. Ejecutar `main.py`, ingresar como `admin` y dar de alta un socio (Menú 1 → Opción 1).
2. Salir con la opción 0.
3. Volver a ejecutar `main.py` e iniciar sesión: el socio aparece en el listado (Menú 1 → Opción 4), recuperado desde `datos_gimnasio.json`.
4. Abrir la opción 7 del menú principal: se ven los eventos `ALTA_SOCIO`, `CIERRE_SESION` y el nuevo `ACCESO_EXITOSO`, numerados en forma continua.

7.5. Contenido de la entrega

Elemento	Ubicación
Código fuente	`main.py` y carpeta `src/`
Archivo de texto	`auditoria.txt`
Archivo JSON	`datos_gimnasio.json`
Pruebas unitarias	`test_gymcontrol.py`
Documentación	`Documentacion/Documentacion_Tecnica_TP2.md` (este documento) y `README.md`